

```
=====
1 INTERVIEW PREP – SOFTWARE ENGINEER, THANE (NETWORK MARVELS)
2 TOMORROW 2 PM | FACE-TO-FACE
3 Samaranjan Manjhi | +91 9082650120
4 =====
5
6
7
8 =====
9 SECTION 0 – TELL ME ABOUT YOURSELF (say this exactly)
10 =====
11
12 "My name is Samaranjan Manjhi. I'm a C/C++ Systems and Security Engineer
13 with 3 years of experience at MicroWorld Software Services in Mumbai.
14 I've built 16 production-grade security modules – covering Linux kernel
15 monitors, macOS endpoint security, DLP engines, binary integrity systems,
16 and firewall rule generators – all integrated into eScan EDR/XDR products.
17
18 My key strengths are kernel-level programming – I've worked with fanotify
19 for execution gating, ftrace for syscall hooking, the macOS Endpoint
20 Security Framework, Ed25519 cryptographic signing, and Unix domain socket IPC.
21
22 I'm a hands-on builder. Every project on my resume is a production module
23 I designed and shipped – not academic work.
24
25 Now I'm looking for a role where I can expand into Windows systems
26 development. My OS internals knowledge is deep – process model, memory
27 management, syscalls, IPC – and I'm confident I can ramp up on Windows
28 SDK and KMDF within 4 to 5 weeks given my Linux kernel background."
29
30
31 =====
32 SECTION 1 – PRIORITY 1 (do this first – biggest risk)
33 =====
34
35 --- YOUR WINDOWS GAP (JD requires Windows SDK + KMDF) ---
36
37 Your answer when they ask about Windows experience:
38 "I don't have hands-on Windows SDK or KMDF experience yet, but my
39 kernel-level fundamentals are very transferable. On Linux I've written
40 kernel modules (.ko), hooked syscalls via ftrace, and used fanotify
41 for execution gating. Windows KMDF uses the same WDM model – DriverEntry
42 is like module_init(), IRPs are like file operation structs, dispatch
43 routines map to my fanotify handler callbacks. I estimate 4-5 weeks to
44 be productive on Windows kernel code."
45
46 Bridge analogies (use these):
47 fanotify FAN_OPEN_PERM ≈ Windows minifilter pre-operation callback
48 Linux .ko DriverEntry ≈ Windows KMDF DriverEntry
49 Unix domain sockets ≈ Windows named pipes
50 inotify ≈ ReadDirectoryChangesW
51 /proc / /sys ≈ Windows Registry / Device I/O Control
```

Windows terms to know (just the terms, not deep):

```
KMDF      = Kernel-Mode Driver Framework
WDM       = Windows Driver Model
IRP       = I/O Request Packet (kernel request object)
DriverEntry = entry point of a Windows driver (like main())
EvtIoRead / EvtIoWrite = KMDF dispatch callbacks
HANDLE    = Windows opaque resource handle (like fd in Linux)
CreateFile = opens files AND devices (not just files)
DeviceIoControl = user-space to kernel communication
WinDBG    = Windows kernel debugger (like GDB for kernel)
OVERLAPPED = async I/O struct (like epoll + callback)
```

.NET gap answer:

"I have no hands-on .NET experience. But C++ OOP and C# OOP are the same principles – classes, inheritance, polymorphism. C# runs on CLR with GC – I actually find that interesting coming from manual memory C++. I can pick up C# quickly. Delegates ≈ std::function, async/await ≈ my epoll-based async I/O."

--- OOP IN C++ (will definitely be asked) ---

Q1. What are the 4 pillars of OOP?

A: 1. Encapsulation – hide data inside classes (private members)
2. Abstraction – expose only what's needed (interfaces/virtual)
3. Inheritance – child reuses parent class
4. Polymorphism – same call, different behavior at runtime

Q2. Virtual function vs pure virtual function?

A: Virtual = has a default body, child CAN override
Pure virtual = "= 0", no body, class is abstract, child MUST override

```
class Shape {
    virtual void draw() {} // can override
    virtual double area() = 0; // must override – Shape is abstract
};
```

Q3. What is vtable?

A: Compiler creates a hidden table of function pointers per class.
When you call a virtual function via base pointer, the runtime looks up the vtable to find the correct derived class function.
This is how polymorphism works under the hood.
Base* b = new Derived(); b->show(); // vtable lookup → calls Derived::show()

Q4. What is RAII? (link to your project!)

A: Resource Acquisition Is Initialization. Tie resource lifetime to object lifetime. Constructor acquires, destructor releases.
Prevents leaks even on exceptions.

```
class FileHandle {
    FILE* f;
public:
```

```
FileHandle(const char* p) { f = fopen(p,"r"); }
```

```
~FileHandle() { if(f) fclose(f); } // auto-close
```

```
};
```

YOUR ANSWER: "In my DLP scanner I use RAII wrappers around SQLite3

handles – they auto-close on any exit path including exceptions."

Q5. Smart pointers – unique_ptr vs shared_ptr vs weak_ptr?

A: unique_ptr = sole ownership, not copyable, zero overhead

shared_ptr = shared ownership via reference count

weak_ptr = non-owning reference to shared_ptr, breaks cycles

When: unique_ptr by default. shared_ptr only when ownership is shared.

weak_ptr to avoid cyclic references (A→B→A would never free).

Q6. Rule of 5?

A: If you define any of: destructor, copy constructor, copy assignment,

move constructor, move assignment – define all 5.

Why: compiler-generated ones do shallow copy → double free on raw pointers.

Q7. Struct vs class in C++?

A: Only difference: default access.

struct = public by default.

class = private by default.

Otherwise identical – both can have constructors, methods, inheritance.

Q8. Diamond problem?

A: B and C both inherit A. D inherits B and C. D has two copies of A.

Fix: virtual inheritance – class B : virtual public A;

Now only one copy of A exists in D.

Q9. Deep copy vs shallow copy?

A: Shallow copy: copies pointer (both point to same memory → double free)

Deep copy: copies the data the pointer points to (independent copy)

Relevant: default copy constructor does shallow copy – dangerous for

classes owning heap memory.

--- OS INTERNALS (universal – same on Linux and Windows) ---

Q1. What happens when exec() is called?

A: 1. Kernel finds the binary on disk

2. Checks permissions + reads ELF header

3. Loads program segments into virtual memory

4. Sets up stack, heap, BSS

5. Transfers control to entry point (_start → main)

YOUR ANSWER: "In MW Guardian, fanotify FAN_OPEN_PERM intercepts

exec() at step 2 – we verify the Ed25519 signature BEFORE the

binary gets to step 3. Tampered binary = SIGKILL."

Q2. Process vs thread?

A: Process: independent address space, own file descriptors, own PID.

Thread: shares address space and file descriptors with other threads

of the same process. Lighter to create and switch.

YOUR ANSWER: "In MW Guardian the fanotify event loop runs on one thread, Unix socket IPC server on another – shared queue protected by mutex."

Q3. What is a system call? How does your ftrace work?

A: Syscall: user-space requests kernel service. CPU switches from ring 3 (user) to ring 0 (kernel), executes, returns.
ftrace: kernel's built-in function tracer. Register a callback that fires before/after any kernel function – including sys_rename.
No kernel recompile needed. Your answer: "I hooked sys_rename via ftrace in my ransomware module. Bulk renames exceeding threshold → SIGKILL offending PID + JSON log to dmesg."

Q4. Virtual memory and page tables?

A: Each process sees a private 64-bit address space. OS maps virtual addresses to physical RAM via page tables. TLB caches recent mappings.
Page fault: virtual address not mapped → OS loads page from disk.

Q5. Stack vs heap?

A: Stack: auto-managed, LIFO, limited size, fast. Local variables.
Heap: manual (malloc/new), unlimited (RAM), slower. Dynamic objects.
Stack overflow: deep recursion. Heap leak: forgot to free/delete.

=====

SECTION 2 – PRIORITY 2 (do after P1)

=====

--- MULTITHREADING & IPC ---

Q1. Mutex vs semaphore?

A: Mutex: ownership – only the locker can unlock. Binary. Mutual exclusion.
Semaphore: no ownership – any thread can signal. Counting (0 to N).
Use semaphore for: limiting concurrent DB connections, signaling events.
Use mutex for: protecting a shared data structure.

Q2. Deadlock – 4 Coffman conditions?

A: 1. Mutual exclusion – resource held by only one thread
2. Hold and wait – holding one resource, waiting for another
3. No preemption – resource can't be taken forcibly
4. Circular wait – A waits for B, B waits for A
Break ANY one condition → no deadlock possible.

Q3. Race condition?

A: Two threads access shared data without synchronization.
Result is non-deterministic. Fix: mutex or atomic operations.

Q4. select vs poll vs epoll?

A: select: limited FDs, slow (O(n) scan)
poll: no FD limit, still O(n) scan

epoll: 0(1) – kernel notifies only active FDs. Used in high-perf servers.

YOUR ANSWER: "In my Content Scanner Unix socket server I used
epoll to handle multiple client connections without blocking."

Q5. condition_variable – when to use?

A: When one thread must wait for a condition set by another thread.

Producer-consumer: consumer waits on cv.wait() until producer pushes.

```
std::unique_lock<std::mutex> lk(mtx);
```

```
cv.wait(lk, [&]{ return !queue.empty(); });
```

--- YOUR PROJECTS – PREPARE 90-SECOND VERBAL SUMMARY FOR EACH ---

MW GUARDIAN (your strongest – expect deep questions):

Problem: binaries can be tampered with after signing → execution of
malicious code even if the file "looks" trusted.

Solution: fanotify FAN_OPEN_PERM freezes the binary AT kernel exec().

Verifier validates Ed25519 signature synchronously while binary is frozen.

Tampered binary gets SIGKILL before a single instruction executes.

Key decisions: Ed25519 (faster than RSA, smaller keys, same security),
multi-key rotation (old keys retained so old binaries stay valid),
self-verification at startup (guardian verifies itself first).

Deep questions you'll get:

Q: What if verifier crashes while binary is frozen?

A: fanotify has a timeout – kernel auto-allows if no response within
the timeout window. Production tweak: restart verifier via systemd
watchdog before timeout.

Q: Why Ed25519 not RSA?

A: Ed25519 verification is ~2x faster than RSA-2048. Key is 32 bytes
vs 256 bytes. Same 128-bit security level. And libsodium makes it
trivial to implement safely.

RANSOMWARE DEFENSE:

Problem: no kernel-level ransomware detection on this Linux system.

Solution: ftrace hooks sys_rename – detects bulk-rename pattern +
suspicious extensions (e.g. .locked, .enc). Threshold hit → SIGKILL PID.

Module hides from lsmod. sysfs interface password-protected.

Q: Is hiding from lsmod a rootkit technique?

A: Yes, it uses the same mechanism – removing module from the kernel
list. Here it's a defensive choice: attacker who finds the module
in dmesg logs cannot easily remove it. Security through defense-in-depth.

CONTENT SCANNER / DLP ENGINE:

Problem: scanning PII in multiple file formats with low latency.

Solution: shared library + standalone CLI. Unix socket server variant
keeps the PCRE2 regex engine initialized – no cold-start cost per scan.
Handles 15+ formats via Poppler (PDF), Tesseract (OCR), libarchive.

Q: Why Unix socket server variant?

A: Initializing PCRE2 engine (compile 30+ patterns) costs ~50ms.

Socket server does it once at startup. Clients send file paths,
get results. Per-scan overhead drops to pure file I/O + match time.

--- APTITUDE ROUND – C/C++ TRICKY OUTPUT QUESTIONS ---

Q. `int arr[]={10,20,30,40}; int *p=arr; printf("%d %d",*p,*(p+2));`

A. Output: 10 30

`p = &arr[0], p+2 = &arr[2]`. Pointer adds `sizeof(int)*2` bytes.

Q. `int a=5; printf("%d %d %d", a++, ++a, a);`

A. UNDEFINED BEHAVIOR – say this explicitly. Multiple mods between sequence
points. Correct answer: "This is UB in C." Shows you know the standard.

Q. `struct A { char c; int i; char d; }; sizeof(A)?`

A. Answer: 12 bytes.

`c(1) + padding(3) + i(4) + d(1) + padding(3) = 12`

Alignment: int must be at 4-byte boundary.

To reduce: put int first → `struct B { int i; char c; char d; } = 8 bytes.`

Q. `printf("%d", sizeof(int));`

A. Technically UB – `sizeof` returns `size_t`, need `%zu` not `%d`.

Say "this should use `%zu`; on most 32/64-bit systems `sizeof(int)` is 4."

Q. Output of: `int x=10; int *p=&x; *p=20; printf("%d",x);`

A. Output: 20. `p` points to `x`, `*p=20` modifies `x` directly.

Q. What does `malloc` return on failure?

A. NULL. Always check: `if(!ptr) { handle_error(); }`

`new` throws `std::bad_alloc` (unless you use `new(nothrow)`).

=====

SECTION 3 – PRIORITY 3 (cover on commute/before interview)

=====

--- DESIGN QUESTION FRAMEWORK (they WILL ask this) ---

"Design a file monitoring system" – YOU BUILT THIS. Say:

1. Requirements: watch paths, detect modify/delete/rename, alert mechanism
2. Data structures: `map<path, FileInfo{hash, size, mtime}>` in SQLite
3. Event source: `FSEvents` (macOS) / `inotify` (Linux) / `ReadDirectoryChangesW` (Windows)
4. Threading: event loop thread + alert dispatch thread
5. Edge cases: atomic renames (rename A→B→A), high-frequency writes
6. Testing: inject known file changes, verify alerts fire within 1 second

"Design a DLP content scanner" – YOU BUILT THIS. Say:

1. Requirements: scan files for PII (PAN, SSN, email, phone, Aadhaar)
2. Input pipeline: detect MIME type → route to PDF/image/text handler

```

307 3. Core engine: PCRE2 compiled patterns (compile once, reuse)
308 4. OCR fallback: Tesseract for images/scanned PDFs
309 5. Output: structured JSON with pattern name, match offset, file path
310 6. Performance: Unix socket server = regex engine stays warm
311
312
313 --- SQL (may be in aptitude or tech round) ---
314
315 Second highest salary (MOST ASKED):
316 SELECT salary FROM (
317     SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
318     FROM employees
319 ) t WHERE rnk = 2;
320 -- Always mention DENSE_RANK – shows you know window functions
321
322 Top 3 users by spending (CONFIRMED ASKED IN NETWORK MARVELS APR 2025):
323 SELECT u.user_id, u.name, SUM(o.amount) AS total_spent
324 FROM users u
325 JOIN orders o ON u.user_id = o.user_id
326 WHERE o.created_at >= NOW() - INTERVAL 6 MONTH
327 GROUP BY u.user_id, u.name
328 ORDER BY total_spent DESC
329 LIMIT 3;
330
331 WHERE vs HAVING:
332 WHERE filters rows BEFORE GROUP BY.
333 HAVING filters groups AFTER GROUP BY.
334 Cannot use SUM/COUNT in WHERE.
335
336 Find duplicates:
337 SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;
338
339 Types of JOINS:
340 INNER JOIN = only matching rows from both
341 LEFT JOIN = all from left + matching from right (NULL if no match)
342 RIGHT JOIN = all from right + matching from left
343 FULL OUTER = all from both, NULL where no match
344
345 Indexing basics:
346 B-tree structure. Speeds up SELECT, slows INSERT/UPDATE. Use on WHERE/JOIN columns.
347 CREATE INDEX idx_email ON users(email);
348
349
350 =====
351 SECTION 4 – DSA CODING (1-2 problems expected)
352 =====
353
354 --- MOST LIKELY PROBLEMS (in order of probability) ---
355
356 1. REVERSE A LINKED LIST (almost certain to be asked)
357 ListNode* reverse(ListNode* head) {

```

```

358     ListNode *prev = nullptr, *curr = head, *next;
359     while(curr) {
360         next = curr->next;
361         curr->next = prev;
362         prev = curr;
363         curr = next;
364     }
365     return prev;
366 }
367 Time O(n), Space O(1). Trace: 1→2→3 becomes 3→2→1
368
369 2. DETECT CYCLE – FLOYD'S ALGORITHM (very likely)
370 bool hasCycle(ListNode* head) {
371     ListNode *slow=head, *fast=head;
372     while(fast && fast->next) {
373         slow = slow->next;
374         fast = fast->next->next;
375         if(slow == fast) return true;
376     }
377     return false;
378 }
379 If asked "where does cycle start": reset slow to head, move both 1 step at a time → they meet at cycle
start.
380
381 3. LONGEST SUBSTRING WITHOUT REPEATING CHARS (likely)
382 int lengthOfLongestSubstring(string s) {
383     unordered_set<char> seen;
384     int left=0, maxLen=0;
385     for(int right=0; right<s.size(); right++) {
386         while(seen.count(s[right]))
387             seen.erase(s[left++]);
388         seen.insert(s[right]);
389         maxLen = max(maxLen, right-left+1);
390     }
391     return maxLen;
392 }
393 Time O(n), Space O(n). "abcabcbb" → 3 ("abc")
394
395 4. TWO SUM (easy warm-up)
396 vector<int> twoSum(vector<int>& nums, int target) {
397     unordered_map<int,int> mp;
398     for(int i=0; i<nums.size(); i++) {
399         int comp = target - nums[i];
400         if(mp.count(comp)) return {mp[comp], i};
401         mp[nums[i]] = i;
402     }
403     return {};
404 }
405 Time O(n). Don't do the O(n2) brute force.
406
407 5. THREAD-SAFE PRODUCER-CONSUMER QUEUE (possible – directly from your work)

```



```

408 template<typename T>
409 class SafeQueue {
410     queue<T> q;
411     mutex mtx;
412     condition_variable cv;
413 public:
414     void push(T val) {
415         unique_lock<mutex> lk(mtx);
416         q.push(val);
417         cv.notify_one();
418     }
419     T pop() {
420         unique_lock<mutex> lk(mtx);
421         cv.wait(lk, [&]{ return !q.empty(); });
422         T val = q.front(); q.pop();
423         return val;
424     }
425 };

```

426 YOUR ANSWER: "This is essentially what I use in MW Guardian for the
427 event queue between the fanotify thread and IPC handler thread."
428

429 6. VALID PARENTHESES – STACK (common in aptitude)

```

430 bool isValid(string s) {
431     stack<char> st;
432     for(char c : s) {
433         if(c=='('||c=='{'||c=='[') st.push(c);
434         else {
435             if(st.empty()) return false;
436             char top=st.top(); st.pop();
437             if(c==')'&&top!='(') return false;
438             if(c=='}'&&top!='{') return false;
439             if(c==']'&&top!='[') return false;
440         }
441     }
442     return st.empty();
443 }
444

```

445 7. BINARY SEARCH ON ROTATED SORTED ARRAY (medium – possible)

```

446 int search(vector<int>& nums, int target) {
447     int lo=0, hi=nums.size()-1;
448     while(lo<=hi) {
449         int mid=(lo+hi)/2;
450         if(nums[mid]==target) return mid;
451         if(nums[lo]<=nums[mid]) { // left half sorted
452             if(nums[lo]<=target && target<nums[mid]) hi=mid-1;
453             else lo=mid+1;
454         } else { // right half sorted
455             if(nums[mid]<target && target<=nums[hi]) lo=mid+1;
456             else hi=mid-1;
457         }
458     }

```

```

459     return -1;
460 }
461
462 8. LEVEL ORDER TRAVERSAL OF BINARY TREE (medium)
463 vector<vector<int>> levelOrder(TreeNode* root) {
464     vector<vector<int>> res;
465     if(!root) return res;
466     queue<TreeNode*> q;
467     q.push(root);
468     while(!q.empty()) {
469         int sz=q.size();
470         vector<int> level;
471         for(int i=0;i<sz;i++) {
472             auto node=q.front(); q.pop();
473             level.push_back(node->val);
474             if(node->left) q.push(node->left);
475             if(node->right) q.push(node->right);
476         }
477         res.push_back(level);
478     }
479     return res;
480 }
481
482 9. FIND MIDDLE OF LINKED LIST (easy)
483 ListNode* findMiddle(ListNode* head) {
484     ListNode *slow=head, *fast=head;
485     while(fast && fast->next) {
486         slow=slow->next;
487         fast=fast->next->next;
488     }
489     return slow;
490 }
491
492 10. REVERSE A STRING IN-PLACE (aptitude)
493 void reverseString(string& s) {
494     int l=0, r=s.size()-1;
495     while(l<r) swap(s[l++], s[r--]);
496 }
497
498 --- TIME COMPLEXITY CHEATSHEET ---
499 Reverse linked list      O(n) time,  O(1) space
500 Floyd's cycle detection  O(n) time,  O(1) space
501 Two sum (hashmap)       O(n) time,  O(n) space
502 Binary search           O(log n) time, O(1) space
503 Sliding window          O(n) time,  O(k) space
504 Level order BFS         O(n) time,  O(n) space (queue)
505 Sorting (std::sort)     O(n log n)
506
507
508 =====
509 SECTION 5 – MAXIMUM FREQUENCY QUESTIONS (asked most in past)

```

510 =====

511
512 Based on your readMe + interview patterns for this company and role:
513

514 #1 CONFIRMED ASKED AT NETWORK MARVELS (APR 2025):
515 SQL – Top 3 users by spending in last 6 months.
516 (Answer in Section 3 above)
517

518 #2 ALMOST CERTAIN (every C++ interview):
519 - Reverse a linked list
520 - Virtual functions and vtable
521 - What is RAII
522 - Deadlock – 4 Coffman conditions
523 - Deep copy vs shallow copy
524

525 #3 HIGHLY LIKELY (systems/security role):
526 - How does fanotify FAN_OPEN_PERM work (from your resume)
527 - Why Ed25519 over RSA
528 - What happens when exec() is called
529 - Mutex vs semaphore
530 - Process vs thread
531

532 #4 COMMON C++ MCQ/OUTPUT TRAPS:
533 - struct padding / sizeof output
534 - pre/post increment UB
535 - pointer arithmetic output
536 - malloc returns NULL on failure (new throws)
537

538 #5 BEHAVIORAL (always asked in HR round):
539 - Why are you leaving MicroWorld?
540 - Tell me about a difficult bug you fixed.
541 - How do you handle learning a new tech fast?
542
543

544 =====

545 SECTION 6 – 3 SMART QUESTIONS TO ASK INTERVIEWER AT END

546 =====

547
548 Ask these in this order – pick the first 3 that fit:
549

550 1. "What does the Windows kernel / driver development stack look like
551 for your product – KMDF or UMDF? And is the codebase primarily C
552 or C++?"
553 (Shows you're already thinking about ramping up)
554

555 2. "What does the first 3 months look like for this role – is there
556 a structured onboarding for the Windows side, or will I be learning
557 from the codebase directly?"
558 (Shows you're being proactive, not just hoping for the job)
559

560 3. "Are you building primarily EDR/DLP type products, or is there

561 kernel driver work on the roadmap?"
562 (Shows genuine interest in the technical direction)
563
564
565 =====
566 SECTION 7 – SALARY TALK
567 =====
568
569 When asked expected CTC:
570 "My current CTC is 5 LPA. Given my 3 years of specialised kernel and
571 security engineering – 16 production modules, kernel-level C/C++ on
572 Linux and macOS – I'm targeting 9 LPA. I understand there'll be a
573 ramp-up on Windows, but the core systems depth transfers immediately."
574
575 If they counter with 6-6.5:
576 "That's below my expectation for this skill set. Is there flexibility?
577 My bottom line is 8 LPA. I'm genuinely interested in the role and
578 what you're building."
579
580 Notice period:
581 Know your exact notice period and state it proactively. It builds trust.
582
583 Commute from Kandivali to Thane:
584 ~50-70 min via Western line to Dadar + Central line to Thane.
585 Be honest if they ask. Don't oversell.
586
587
588 =====
589 SECTION 8 – QUICK REFERENCE – JUST BEFORE INTERVIEW
590 =====
591
592 RAII = tie resource to object lifetime, destructor frees
593 vtable = table of function pointers for virtual dispatch
594 unique_ptr = single owner, no copy, zero overhead
595 shared_ptr = ref-counted shared ownership
596 KMDF = Windows Kernel Mode Driver Framework
597 IRP = Windows I/O Request Packet
598 FAN_OPEN_PERM = fanotify perm event that freezes binary at exec()
599 Ed25519 = elliptic curve signature, 32-byte key, fast verify
600 ftrace = Linux kernel function tracer, no kernel patch needed
601 PCRE2 = Perl-compatible regex engine (your DLP uses this)
602 epoll = Linux async event notification, O(1) for active FDs
603 4 OOP pillars = Encapsulation, Abstraction, Inheritance, Polymorphism
604 4 deadlock cond = Mutex exclusion, Hold&Wait, No preemption, Circular wait
605
606 =====
607 ALL THE BEST TOMORROW – YOU'VE BUILT THE REAL STUFF.
608 THE RESUME IS STRONG. OWN IT.
609 =====
610